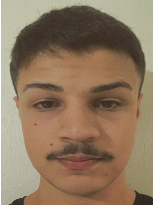


Trabajo Práctico N° 3

Base de datos

Grupo 05

Foto	Apellido y Nombre	Correo electrónico
	Thoux, Ivan Ezequiel	thouxivan@gmail.com
	Thoux, Lucas Andres	luksthoux@hotmail.com
	Krauss, Leandro Ezequiel	kraussleandro598@gmail.com
	Preste, Jose Orlando	josepreste11@gmail.com

Año 2026

Profesores:

- Pautsch, Germán.-
- Yauny, Gustavo.-

Tema: Estructuras de Control: T-SQL .- Cursores - Transacciones

F-) Instrucciones con Estructuras de Control Transact-SQL (IF, ELSE,...DECLARE, SET,)

f.1-) Modificar los SP de tps anteriores, para que implementen los controles necesarios de integridad (verificar que no exista el registro, generar un siguiente nro de Id, etc.) antes de Realizar las Operaciones de (Ins, Upd, Del), y generar Msg de error, capturar err o datos en parámetros output (salida en el SP), lanzar excepciones (msg), (err) según error capturado / generado, invocar / ejecutar otro SP anidado, etc.- (utilizar estructuras de control de T-SQL).-

```
CREATE PROCEDURE SP_Log_Errores
```

```
    @Procedimiento VARCHAR(100),
```

```
    @NumError INT,
```

```
    @MensajeError VARCHAR(4000)
```

```
AS
```

```
BEGIN
```

```
    PRINT '>> LOG DE SISTEMA GUARDADO: Error ' + CAST(@NumError AS  
    VARCHAR) + ' en ' + @Procedimiento + ' - Detalle: ' + @MensajeError;
```

```
END;
```

```
GO
```

```
CREATE PROCEDURE SP_AltaEmpleado_Avanzado
```

```
    @Apellido VARCHAR(30),
```

```
    @Nombre VARCHAR(30),
```

```
    @CodCateg INT,
```

```
    @NrSuc INT,
```

```
    @CodCiud INT,
```

```
    @LegajoGenerado_OUT INT OUTPUT
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    BEGIN TRY
```

```
        IF NOT EXISTS (SELECT 1 FROM SUCURSALES WHERE NrSuc = @NrSuc)
```

```
            THROW 51001, 'Error de Integridad: La sucursal indicada no existe en el sistema.',
```

```
1;
```

```
        IF NOT EXISTS (SELECT 1 FROM CATEGORIAS WHERE CodCateg =  
        @CodCateg)
```

```

        THROW 51002, 'Error de Integridad: El código de categoría ingresado es inválido.',
1;

    DECLARE @NuevoLegajo INT;

    SELECT @NuevoLegajo = ISNULL(MAX(LegEmp), 0) + 1 FROM EMPLEADOS;

    INSERT INTO EMPLEADOS (LegEmp, Apellido, Nombre, CodCateg, NrSuc,
CodCiud, FechaIngreso)

        VALUES (@NuevoLegajo, @Apellido, @Nombre, @CodCateg, @NrSuc,
@CodCiud, GETDATE());

    SET @LegajoGenerado_OUT = @NuevoLegajo;

    PRINT 'Operación Exitosa: Empleado ' + @Apellido + ' registrado. Legajo Asignado: '
+ CAST(@NuevoLegajo AS VARCHAR);

END TRY

BEGIN CATCH

    DECLARE @ErrorNum INT = ERROR_NUMBER();

    DECLARE @ErrorMsg VARCHAR(4000) = ERROR_MESSAGE();

    PRINT '=== ATENCIÓN: LA OPERACIÓN HA FALLADO ===';

    EXEC SP_Log_Errores 'SP_AltaEmpleado_Avanzado', @ErrorNum, @ErrorMsg;

    THROW;

END CATCH

END;

GO

```

f.2-) Escribir los SP con **Instrucciones T-SQL** necesarios para realizar y garantizar la correcta y completa operatoria de un Proceso de TRANSFERENCIA entre 2 CUENTAS (controlando y previendo absolutamente todo),

SP_DEBITO (cuenta origen, monto, fecha, codOp, etc.)

SP_CREDITO (cuenta destino, monto, fecha, codOp, etc.)

SP_TRANSFERENCIA (cuenta_origen, Cuenta_destino, monto, fecha, codOp1, CodOp2, .. etc.)

Considerar:

- SP Completamente Parametrizado
- Controlar existencia de ambas cuentas
- Si el Saldo es suficiente - Garantizar con transacciones las operaciones de débito / crédito por separado

- Registrar los Movimientos - Generar el registro de Auditoría
- Verificar valores variable @error
- Invocar internamente la ejecución de otros SP, como (SP_DEBITO, SP_CREDITO), según corresponda el caso, Etc.

SP_DEBITO

```
CREATE OR REPLACE PROCEDURE sp_debito(
  p_cuenta INT,
  p_monto NUMERIC,
  p_fecha DATE,
  p_codop INT
)
LANGUAGE plpgsql AS $$
DECLARE
  v_saldo_actual NUMERIC;
  v_nrmov INT;
BEGIN
  SELECT saldo::numeric INTO v_saldo_actual
  FROM cuentas
  WHERE nrcuenta = p_cuenta
  FOR UPDATE;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'La cuenta origen % no existe.', p_cuenta;
  END IF;

  IF v_saldo_actual < p_monto THEN
    RAISE EXCEPTION 'Saldo insuficiente en la cuenta origen %.', p_cuenta;
  END IF;

  UPDATE cuentas
  SET saldo = saldo - p_monto
  WHERE nrcuenta = p_cuenta;

  SELECT COALESCE(MAX(nrmov), 0) + 1 INTO v_nrmov FROM movimientos;
  INSERT INTO movimientos(nrmov, nrcuenta, monto, fecha, codop)
  VALUES (v_nrmov, p_cuenta, p_monto::money, p_fecha, p_codop);
END;
$$;
```

SP_CREDITO

```
CREATE OR REPLACE PROCEDURE sp_credito(
  p_cuenta INT,
  p_monto NUMERIC,
  p_fecha DATE,
```

```

    p_codop INT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_nrmov INT;
BEGIN
    IF NOT EXISTS (SELECT 1 FROM cuentas WHERE nrcuenta = p_cuenta FOR
UPDATE) THEN
        RAISE EXCEPTION 'La cuenta destino % no existe.', p_cuenta;
    END IF;

    UPDATE cuentas
    SET saldo = saldo + p_monto
    WHERE nrcuenta = p_cuenta;

    SELECT COALESCE(MAX(nrmov), 0) + 1 INTO v_nrmov FROM movimientos;
    INSERT INTO movimientos(nrmov, nrcuenta, monto, fecha, codop)
    VALUES (v_nrmov, p_cuenta, p_monto::money, p_fecha, p_codop);
END;
$$;

```

SP_TRANSFERENCIA

```

CREATE OR REPLACE PROCEDURE sp_transferencia(
    p_cuenta_origen INT,
    p_cuenta_destino INT,
    p_monto NUMERIC,
    p_fecha DATE,
    p_codop1 INT,
    p_codop2 INT
)
LANGUAGE plpgsql AS $$
BEGIN
    CALL sp_debito(p_cuenta_origen, p_monto, p_fecha, p_codop1);
    CALL sp_credito(p_cuenta_destino, p_monto, p_fecha, p_codop2);

    COMMIT;
    RAISE NOTICE 'Transferencia de $% realizada con éxito. Origen: %, destino: %',
p_monto, p_cuenta_origen, p_cuenta_destino;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE NOTICE 'Error en la transferencia. Operación cancelada. Detalle técnico: %',
SQLERRM;
END;
$$;

```

f.3) Implemente mediante CASE, WHEN

f.3.a-) Un sp con parámetro de entrada: nr cuenta, y que devuelva un mensaje diciendo de qué tipo es la cuenta, en base al “tipo de cuenta”: cta. Cte, caja ahorro, realizando la selección mediante CASE

```
CREATE OR REPLACE PROCEDURE sp_tipo_cuenta_mensaje(p_nrcuenta INT)
LANGUAGE plpgsql AS $$
DECLARE
    v_tipo INT;
    v_mensaje VARCHAR;
BEGIN
    SELECT tipocuenta INTO v_tipo
    FROM cuentas
    WHERE nrcuenta = p_nrcuenta;

    IF NOT FOUND THEN
        RAISE NOTICE 'La cuenta número % no se encuentra en el sistema.', p_nrcuenta;
        RETURN;
    END IF;

    v_mensaje := CASE v_tipo
        WHEN 1 THEN 'Caja de ahorro'
        WHEN 2 THEN 'Cuenta corriente'
        WHEN 3 THEN 'Cuenta sueldo'
        ELSE 'Tipo de cuenta especial/desconocido'
    END;

    RAISE NOTICE 'La cuenta % está clasificada como: %', p_nrcuenta, v_mensaje;
END;
$$;
```

f.3.b-) Realizar un SP, que clasifique las cuentas por su saldo en alguna de las 3 posibilidades: “saldo bajo” < de 10.000 \$, “saldo medio” : entre 10.000 y 50.000, y/o “saldo alto”: > a 50.000), informando con dichos mensajes, en base al rango donde se encuadre el saldo de cada cuenta. respectivamente.-

```
CREATE OR REPLACE FUNCTION fn_clasificar_cuentas_saldo()
RETURNS TABLE (
    nro_cuenta INT,
    saldo_actual NUMERIC,
    clasificacion VARCHAR
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT
        c.nrcuenta,
```

```

c.saldo::numeric,
CASE
  WHEN c.saldo::numeric < 10000 THEN 'Saldo bajo'::VARCHAR
  WHEN c.saldo::numeric BETWEEN 10000 AND 50000 THEN 'Saldo
medio'::VARCHAR
  WHEN c.saldo::numeric > 50000 THEN 'Saldo alto'::VARCHAR
END
FROM cuentas c;
END;
$$;

```

G-) Implementar las soluciones a los siguientes casos

Realizarlo en algún tipo de objeto que considere (vista, proced. Almacenado, disparador), utilizando las instrucciones y estructuras de control y utilizando las variables del sistema @, o @@ o definiendo las propias, como también los parámetros de entrada y salida:

g.1 - Actualizar el saldo de una cuenta en cierto %.

```

CREATE PROCEDURE SP_ActualizarSaldo_Porcentaje (
  @NroCuenta INT,
  @Porcentaje DECIMAL(5,2)
)
AS
BEGIN
  -- Se actualiza la cuenta sumándole el % indicado por parámetro
  UPDATE Cuentas
  SET saldo = saldo + (saldo * (@Porcentaje / 100))
  WHERE nro_cuenta = @NroCuenta;
END
GO

```

g.2 - Modificar el domicilio de un cliente x.

```

CREATE PROCEDURE SP_ModificarCiudadCliente
(
  @DNI INT,
  @NuevoCodCiud INT
)
AS
BEGIN
  IF NOT EXISTS(
    SELECT 1
    FROM CLIENTES
    WHERE DNI = @DNI

```

```

)
BEGIN
    RAISERROR('Cliente inexistente.',16,1);
    RETURN;
END
UPDATE CLIENTES
SET CodCiud = @NuevoCodCiud
WHERE DNI = @DNI;
PRINT 'Ciudad actualizada correctamente.';
END
GO

```

g.3 - Obtener todas las cuentas pertenecientes a un cliente x.

```
CREATE PROCEDURE ObtenerCuentasPorCliente
```

```
    @DNI_Cliente INT,
```

```
    @CantidadCuentas INT OUTPUT
```

```
AS
```

```
BEGIN
```

```
    BEGIN TRY
```

```
        SELECT
```

```
        NrCuenta,
```

```
        Saldo,
```

```
        NrSuc
```

```
        FROM
```

```
        CUENTAS
```

```
        WHERE
```

```
        DNI = @DNI_Cliente;
```

```
        SET @CantidadCuentas = @@ROWCOUNT;
```

```
        IF @CantidadCuentas = 0
```

```
            BEGIN
```

```
                PRINT 'El cliente con DNI ' + CAST(@DNI_Cliente AS VARCHAR) + ' no posee
cuentas registradas.';
```

```
            END
```

```
        ELSE
```

```
            BEGIN
```

```

        PRINT 'Se encontraron ' + CAST(@CantidadCuentas AS VARCHAR) + ' cuenta(s)
para el cliente.';

    END

    END TRY

    BEGIN CATCH

        PRINT 'Ocurrió un error al ejecutar la consulta. Código de error: ' +
CAST(@@ERROR AS VARCHAR);

    END CATCH

END;

```

g.4 - Obtener la suma de los saldos de todas las cuentas de un cliente x.

```

CREATE PROCEDURE SP_ObtenerSumaSaldos_Cliente (
    @IdCliente INT,
    @SumaTotalSaldos DECIMAL(18,2) OUTPUT
)
AS
BEGIN
    -- Calculamos la sumatoria y la asignamos a la variable de salida
    SELECT @SumaTotalSaldos = SUM(saldo)
    FROM Cuentas
    WHERE id_cliente = @IdCliente;

    -- Manejo por si el cliente no tiene cuentas y la suma da NULL
    IF @SumaTotalSaldos IS NULL
    BEGIN
        SET @SumaTotalSaldos = 0;
    END
END
GO

```

g.5 - Obtener el nombre de la ciudad y provincia donde se encuentra la sucursal que tiene registrada la cuenta nr. X.

```

CREATE OR REPLACE FUNCTION
fn_obtener_ubicacion_sucursal_cuenta(p_nrcuenta INT)
RETURNS TABLE (
    ciudad_sucursal VARCHAR,
    provincia_sucursal VARCHAR
)

```

```

LANGUAGE plpgsql AS $$
BEGIN
  RETURN QUERY
  SELECT ci.ciudad::VARCHAR, pr.provincia::VARCHAR
  FROM cuentas cu
  JOIN sucursales s ON cu.nrsuc = s.nrsuc
  JOIN ciudades ci ON s.codciud = ci.codciud
  JOIN provincias pr ON ci.codprov_provincias = pr.codprov
  WHERE cu.nrcuenta = p_nrcuenta;
END;
$$;

```

g.6 - Calcular y mostrar el monto total (o suma) , en concepto de extracciones realizadas sobre una cuenta x, durante los últimos n días.

```
CREATE PROCEDURE CalcularTotalExtracciones
```

```
  @Cuenta_X INT, @Dias_N INT, @MontoTotal DECIMAL(18,2) OUTPUT
```

```
AS
```

```
BEGIN
```

```
  BEGIN TRY
```

```
    IF NOT EXISTS (SELECT 1 FROM CUENTAS WHERE NrCuenta = @Cuenta_X)
```

```
      BEGIN
```

```
        PRINT 'Atención: La cuenta Nro ' + CAST(@Cuenta_X AS VARCHAR) + ' no existe.';
```

```
        SET @MontoTotal = 0; RETURN;
```

```
      END
```

```
        SELECT @MontoTotal = SUM(ABS(Monto)) FROM MOVIMIENTOS WHERE
        NrCuenta = @Cuenta_X AND Monto < 0 AND Fecha >= DATEADD(DAY, -@Dias_N,
        GETDATE());
```

```
        IF @MontoTotal IS NULL
```

```
          BEGIN
```

```
            SET @MontoTotal = 0; PRINT 'No se registraron extracciones en los últimos ' +
            CAST(@Dias_N AS VARCHAR) + ' días.';
```

```
          END
```

```
        ELSE
```

```
          BEGIN
```

```
            PRINT 'Cálculo exitoso. (Filas afectadas: ' + CAST(@@ROWCOUNT AS
            VARCHAR) + ')'; PRINT 'Monto total extraído: $' + CAST(@MontoTotal AS VARCHAR);
```

```
          END
```

```
        END TRY
```

```
        BEGIN CATCH
```

```

PRINT 'Error (' + CAST(@@ERROR AS VARCHAR) + '): ' + ERROR_MESSAGE();
END CATCH
END;

```

g.7 - Obtener los últimos n movimientos sobre la cuenta x, agrupados por tipoOp.

```

CREATE PROCEDURE SP_UltimosMovimientosPorTipo
(
    @NrCuenta INT,
    @Cantidad INT
)
AS
BEGIN

    IF NOT EXISTS(
        SELECT 1
        FROM CUENTAS
        WHERE NrCuenta = @NrCuenta
    )
    BEGIN
        RAISERROR('La cuenta no existe.',16,1);
        RETURN;
    END

    SELECT
        CodOp,
        COUNT(*) AS CantidadMovimientos,
        SUM(Monto) AS TotalOperado
    FROM
    (
        SELECT TOP (@Cantidad)
            CodOp,
            Monto,
            Fecha
        FROM MOVIMIENTOS
        WHERE NrCuenta = @NrCuenta
        ORDER BY Fecha DESC
    ) X
    GROUP BY CodOp;

END
GO

```

g.8 - Incrementar en un valor x%, el sueldo de los empleados en base a su categoría.

```

CREATE PROCEDURE IncrementarSueldoPorCategoria
    @Porcentaje DECIMAL(5,2), @Categoria VARCHAR(30)
AS

```

```

BEGIN
    BEGIN TRY
        UPDATE EMPLEADOS SET Sueldo = Sueldo * (1 + (@Porcentaje / 100)) WHERE
Categoria = @Categoria;
        IF @@ROWCOUNT = 0
        BEGIN
            PRINT 'No se encontraron empleados en la categoría: ' + @Categoria;
        END
        ELSE
        BEGIN
            PRINT 'Actualización exitosa. Empleados afectados: ' + CAST(@@ROWCOUNT AS
VARCHAR);
        END
        END TRY
        BEGIN CATCH
            PRINT 'Error (' + CAST(@@ERROR AS VARCHAR) + '): ' + ERROR_MESSAGE();
        END CATCH
    END;

```

- Implementar todo sobre la Base de datos, entregar el archivo de datos, backup y log.
- Investigue: implementación y uso de CURSORES en consultas SQL en SGBDR, y resuelva:

H-) Transacciones

h.1-) Implementar un Modelo Transaccional (Begin, Commit, Rollback) sobre 2 Stores Procedures: (SP_Debito y SP_Credito), que aseguren cada uno la completa ejecución de las operaciones de: Débito y Crédito, respectivamente, y todo su control asociado.- con parámetros, control errores, etc.

SP_Debito

```

CREATE PROCEDURE SP_DEBITO
(
    @NrCuenta INT,
    @Monto MONEY
)
AS
BEGIN
    BEGIN TRY

```

```

BEGIN TRANSACTION;

DECLARE @Saldo MONEY;

SELECT @Saldo = Saldo
FROM CUENTAS
WHERE NrCuenta = @NrCuenta;

IF @Saldo IS NULL
    RAISERROR('Cuenta inexistente.',16,1);

IF @Saldo < @Monto
    RAISERROR('Saldo insuficiente.',16,1);

UPDATE CUENTAS
SET Saldo = Saldo - @Monto
WHERE NrCuenta = @NrCuenta;

DECLARE @NrMov INT;

SELECT @NrMov = ISNULL(MAX(NrMov),0)+1
FROM MOVIMIENTOS;

INSERT INTO MOVIMIENTOS
(
    NrMov,
    NrCuenta,
    Monto,
    Fecha,
    CodOp
)
VALUES
(
    @NrMov,
    @NrCuenta,
    @Monto,
    GETDATE(),
    0
);

COMMIT TRANSACTION;

END TRY

BEGIN CATCH

    ROLLBACK TRANSACTION;

    PRINT ERROR_MESSAGE();

END CATCH

END

```

GO

SP_Credito

```
CREATE PROCEDURE SP_CREDITO
(
    @NrCuenta INT,
    @Monto MONEY
)
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;

        IF NOT EXISTS(
            SELECT 1
            FROM CUENTAS
            WHERE NrCuenta = @NrCuenta
        )
        BEGIN
            RAISERROR('Cuenta inexistente.',16,1);
        END

        UPDATE CUENTAS
        SET Saldo = Saldo + @Monto
        WHERE NrCuenta = @NrCuenta;

        DECLARE @NrMov INT;

        SELECT @NrMov = ISNULL(MAX(NrMov),0)+1
        FROM MOVIMIENTOS;

        INSERT INTO MOVIMIENTOS
        (
            NrMov,
            NrCuenta,
            Monto,
            Fecha,
            CodOp
        )
        VALUES
        (
            @NrMov,
            @NrCuenta,
            @Monto,
            GETDATE(),
            1
        );

        COMMIT TRANSACTION;
```

```

END TRY

BEGIN CATCH

    ROLLBACK TRANSACTION;

    PRINT ERROR_MESSAGE();

END CATCH

END
GO

```

h.2-) luego, implementar el modelo Transaccional sobre el SP: SP_TRANSFERENCIA, evaluando donde y como corresponde disponer las instrucciones transaccionales. (considere que este sp invoca a los 2 anteriores)

```

CREATE PROCEDURE SP_TRANSFERENCIA (
    @Cuenta_Origen INT,
    @Cuenta_Destino INT,
    @Monto DECIMAL(18,2),
    @Fecha DATE,
    @CodOp1 INT,
    @CodOp2 INT
)
AS
BEGIN
    BEGIN TRY
        -- Iniciamos el bloque transaccional general
        BEGIN TRANSACTION;

        -- Invocamos la operación de débito a la cuenta origen
        EXEC SP_DEBITO @Cuenta_Origen, @Monto, @Fecha, @CodOp1;

        -- Invocamos la operación de crédito a la cuenta destino
        EXEC SP_CREDITO @Cuenta_Destino, @Monto, @Fecha, @CodOp2;

        -- Si ambas ejecuciones fueron exitosas, confirmamos la transacción
        COMMIT TRANSACTION;

        PRINT 'Transferencia realizada y confirmada con éxito.';
    
```

```

END TRY
BEGIN CATCH
    -- En caso de fallar cualquiera de los dos SP, deshacemos todo el proceso
    IF @@TRANCOUNT > 0
    BEGIN
        ROLLBACK TRANSACTION;
    END

    -- Emitimos un mensaje o capturamos el error generado
    PRINT 'Error en la transferencia. Operación cancelada. Detalle: ' +
    ERROR_MESSAGE();
END CATCH
END
GO

```

I-) CURSORES: Resuelva mediante el uso de CURSORES: (dentro de un SP)

i.1-) En base a la categoría del Empleado, actualizar su sueldo en un x% (+), en base al siguiente detalle: caj -> 10%, adm -> 15% y ger-> 12%, (éstos 6 datos se ingresan como parámetros al sp)

```

CREATE PROCEDURE SP_ActualizarSueldosCategorias
(
    @PorcAdm DECIMAL(5,2),
    @PorcGer DECIMAL(5,2),
    @PorcSeg DECIMAL(5,2)
)
AS
BEGIN

    DECLARE @CodCateg INT;
    DECLARE @Categ VARCHAR(30);

    DECLARE CUR_CATEG CURSOR FOR
    SELECT CodCateg, Categ
    FROM CATEGORIAS;

    OPEN CUR_CATEG;

    FETCH NEXT FROM CUR_CATEG
    INTO @CodCateg, @Categ;

    WHILE @@FETCH_STATUS = 0
    BEGIN

        IF @Categ = 'Administrativo'
        BEGIN

```

```

        UPDATE CATEGORIAS
        SET Sueldo = Sueldo + (Sueldo * @PorcAdm / 100)
        WHERE CodCateg = @CodCateg;
    END

    ELSE IF @Categ = 'Gerente'
    BEGIN
        UPDATE CATEGORIAS
        SET Sueldo = Sueldo + (Sueldo * @PorcGer / 100)
        WHERE CodCateg = @CodCateg;
    END

    ELSE IF @Categ = 'Seguridad'
    BEGIN
        UPDATE CATEGORIAS
        SET Sueldo = Sueldo + (Sueldo * @PorcSeg / 100)
        WHERE CodCateg = @CodCateg;
    END

    FETCH NEXT FROM CUR_CATEG
    INTO @CodCateg, @Categ;

    END

    CLOSE CUR_CATEG;
    DEALLOCATE CUR_CATEG;

END
GO

```

i.2-) – otorgar un premio del 5% de intereses (+), al saldo de las cuentas que sean del tipo: PF (Plazo Fijo), y cuyo saldo sea mayor de \$50,000.00 a la fecha de hoy.

```
CREATE PROCEDURE SP_PremiarCuentasPF
```

```
AS
```

```
BEGIN
```

```
-- Declaración de variables para almacenar los datos del cursor
```

```
DECLARE @NroCuenta INT;
```

```
DECLARE @Saldo DECIMAL(18,2);
```

```
-- Declaración del cursor
```

```
DECLARE cur_CuentasPF CURSOR FOR
```

```
SELECT nro_cuenta, saldo
```

```
FROM Cuentas
```

```
WHERE tipo_cuenta = 'PF' AND saldo > 50000.00;
```

```

-- Apertura del cursor
OPEN cur_CuentasPF;

-- Lectura del primer registro
FETCH NEXT FROM cur_CuentasPF INTO @NroCuenta, @Saldo;

-- Bucle para recorrer mientras haya registros (@@FETCH_STATUS = 0)
WHILE @@FETCH_STATUS = 0
BEGIN
    -- Actualización aplicando el 5% de premio
    UPDATE Cuentas
    SET saldo = saldo + (saldo * 0.05)
    WHERE nro_cuenta = @NroCuenta;

    -- Lectura del siguiente registro
    FETCH NEXT FROM cur_CuentasPF INTO @NroCuenta, @Saldo;
END

-- Cierre y liberación del cursor en memoria
CLOSE cur_CuentasPF;
DEALLOCATE cur_CuentasPF;
END
GO

```